

Instituto Tecnológico de Costa Rica
Escuela de Computación
IC-5701 Compiladores e Intérpretes GR 60

Documentación Proyecto 2

Profesor:

Allan Rodríguez Dávila

Trabajo elaborado por:

Brasly Villarebia Morales - 2023105915
Sebastián Rodríguez Sánchez - 2023074446

17 de enero del 2025

Índice

Manual de usuario	3
1.1 Requisitos del sistema	3
1.2 Estructura del proyecto	3
1.3 Compilación	4
1.4 Flujo de ejecución	4
Pruebas de funcionalidad	5
Descripción del problema	6
Diseño del programa	6
Librerías usadas	9
Análisis de resultados	9
Bitácora	9

Manual de usuario

Este manual describe los pasos necesarios para compilar, ejecutar y utilizar el sistema desarrollado, el cual implementa un analizador léxico y sintáctico para un lenguaje imperativo orientado a la configuración de chips, utilizando JFlex y Java CUP.

1.1 Requisitos del sistema

- **Sistema operativo:** Windows, Linux o macOS.
- **Lenguaje:** Java JDK 8 o superior.
- **Herramientas:**
 - JFlex 1.9.1
 - Java CUP
 - Git (para la bitácora)

1.2 Estructura del proyecto

PP2_SebastianRodriguez_BraslyVillarebia.zip

- info.txt
- documentacion/
 - Documentacion_Externa2.pdf
- Programa/
 - librerias/
 - java-cup-11b-runtime.jar
 - java-cup-11b.jar
 - jflex-full-1.9.1.jar
 - Scripts/
 - Lexer.class — clase compilada del analizador léxico
 - Lexer.flex — archivo fuente del analizador léxico (JFlex)
 - Lexer.java — código generado por JFlex
 - Lexer.java~ — código generado por JFlex
 - Main.class — Clase principal del sistema que coordina la ejecución del análisis léxico y sintáctico.
 - Main.java — Clase principal del sistema que coordina la ejecución del análisis léxico y sintáctico.
 - Main\$1.class — Clase interna generada por el compilador de Java.
 - Nodo.class — Implementación de la estructura del árbol sintáctico (AST).
 - Nodo.java — Implementación de la estructura del árbol sintáctico (AST).
 - Parser.class — Código generado y compilado del analizador sintáctico.
 - Parser.cup — Definición de la gramática del lenguaje en Java CUP.
 - Parser.java — Código generado y compilado del analizador sintáctico.
 - Parser\$CUP\$Parser\$actions.class —
 - pruebas.txt — Archivo de entrada utilizados para pruebas del lenguaje.
 - pruebas2.txt — Archivo de entrada utilizados para pruebas del lenguaje.
 - sym.class — Definición y compilación de los símbolos (tokens) utilizados por el parser.

- sym.java — Definición y compilación de los símbolos (tokens) utilizados por el parser.
- TablaSimbolos.class — Implementación de la tabla de símbolos con manejo de scopes.
- TablaSimbolos.java — Implementación de la tabla de símbolos con manejo de scopes.
- TablaSimbolos\$SymbolInfo.class — Clase interna que representa la información de cada símbolo.
- TablaSimbolos\$SymbolTable.class — Clase interna que representa cada tabla de símbolos por scope.
- tokens.txt — Archivo de salida generado por el análisis léxico.

1.3 Compilación

1.3.1 Generación del Analizador Léxico, Desde la carpeta Scripts, ejecutar: `java -jar ../Librerias/jflex-full-1.9.1.jar Lexer.flex`. Es vital ya que genera el Lexer.java (lo que reconoce los tokens del lenguaje).

1.3.2 Generación del Analizador Sintáctico, Para este punto se usa el comando `java -jar ../Librerias/java-cup-11b.jar -parser Parser Parser.cup`. El cual genera el Parser.java y el sym.java (anteriormente explicados).

1.3.3 Compilación del proyecto, Para poder compilarlo se usa `javac -cp " ../Librerias/" *.java` (para Linux/macOS) y para Windows `javac -cp ".;../Librerias/" *.java`.

1.3.4 Ejecución del programa, Se ejecuta la prueba con: `java -cp " ../Librerias/" Main pruebas2.txt`

1.4 Flujo de ejecución

1. Se lee el archivo fuente (pruebas2.txt)
2. Se ejecuta el analizador léxico.
3. Se ejecuta el analizador sintáctico.
4. Se construye:
 - Tabla de símbolos con manejo de scopes
 - Árbol sintáctico

Se reportan errores léxicos, sintácticos y semánticos.

Se imprimen las tablas de símbolos y el árbol sintáctico.

Pruebas de funcionalidad

Prueba de funcionamiento en caso de no haber ningún error léxico ni gramatical

```
Programa > Scripts > Pruebas2.txt
1  coal navidad ¿ ? ;
2  |   local int x = 5   endl
3  |   decide of
4  |   |   5 > 3 -> ;
5  |   |   |   show 1 endl
6  |   |   !
7  |   end decide endl
8  |   return 3 + 5 endl
9  !
10
```

Impresión de la tabla y el árbol sintáctico

```
braslyvm@VICTUS-HP:~/Escritorio/CompiladoresJFlex/Programa/Scripts$ java -cp ".../Librerias/**" Main Pruebas2.txt
Scope: Main
  x : int (local)
  Tabla de simbolos global
Scope: GLOBAL

Arbol sintactico:
Padre: PROGRAMA
Hijos:
  GLOBALES
  FUNCIONES
  MAIN
Nodo: GLOBALES
Nodo: FUNCIONES
Padre: MAIN
Hijos:
  BLOQUE
Padre: BLOQUE
Hijos:
  SENTENCIAS
Padre: SENTENCIAS
Hijos:
  DECL_LOCAL_ASIG
  DECIDE
  RETURN
Padre: DECL_LOCAL_ASIG
Hijos:
  int
  x
  5
Nodo: int
Nodo: x
Nodo: 5
Padre: DECIDE
Hijos:
  condicion
Padre: condicion
Hijos:
  >
  ->
  BLOQUE
Padre: >
Hijos:
  5
  3
Nodo: 5
Nodo: 3
Nodo: ->
Padre: BLOQUE
Hijos:
  SENTENCIAS
Padre: SENTENCIAS
Hijos:
```

Prueba cuando el archivo fuente no cumple con la gramática y los mensajes de error.

```
Programa > Scripts > Pruebas2.txt
1  coal navidad ¿ ? ¡
2  |   local int x = 5
3  |   decide of
4  |       5 > 3 -> ¡
5  |       |   show 1 endl
6  |       !
7  |   end decide endl
8  |   return 3 + 5
9  |   !
10
```

```
braslyvm@VICTUS-HP:~/Escritorio/CompiladoresJFlex/Programa/Scripts$ java -cp "...\Librerias/*" Main Pruebas2.txt
Error de sintaxis: decide Línea 3 Columna 5
Error de sintaxis: ! Línea 9 Columna 1

El archivo fuente no se puede generar: tiene 2 errores sintáctico.
Análisis sintáctico completado.
```

Descripción del problema

Este segundo entregable, corresponde al “Análisis Sintáctico”, cuyo objetivo es implementar un parser utilizando Java CUP, basado en el analizador léxico desarrollado en el primer entregable.

El sistema debe validar si un archivo fuente:

- Puede ser generado por la gramática definida, y si no, reportarlo.
- Cumple con las reglas sintácticas del lenguaje.
- Maneja correctamente errores léxicos y sintácticos (reflejarlos en consola).
- Construye el árbol sintáctico, mostrando todos los nodos.
- Gestiona tablas de símbolos con scopes.

Diseño del programa

Se usó un diseño modular, en el que se separaron funcionalidades en clases auxiliares, para poder controlar mejor la lógica, y liberar código del cup.

Lexer.flex

El analizador léxico fue desarrollado con **JFlex** y es responsable de transformar el archivo fuente en una secuencia de tokens.

Define expresiones regulares, entre las cuales están:

- **Palabras reservadas del lenguaje**, como world, local, gift, navidad, decide, loop, for, return, show, get, entre otras.
- **Operadores aritméticos**, incluyendo binarios (+, -, *, /, //, %, ^) y unarios (++ , --).
- **Operadores relacionales y lógicos**, tales como >, <, >=, <=, ==, !=, @, ~ y Σ.
- **Delimitadores y símbolos especiales**, como bloques (¡, !), paréntesis (¿, ?), arreglos ([,]), coma, flecha (->) y la palabra endl.
- **Literales** de tipo entero, flotante, booleano, carácter y cadena.
- **Identificadores válidos**, permitiendo letras, números y guiones bajos.

- **Comentarios de una línea** (!) y **comentarios multilínea** (e ... e), los cuales son ignorados durante el análisis.

Se utilizan las directivas %line y %column para poder controlar la posición de cada token (información vista en clase).

Además, implementa **manejo de errores léxicos** mediante la regla comodín [^], que reporta el carácter inválido junto con su línea y columna, permitiendo continuar el análisis (recuperación en modo pánico).

Parser.cup

El analizador sintáctico fue implementado usando CUP.

Entre sus responsabilidades principales se incluyen:

- Definición de terminales y no terminales, alineados directamente con los tokens generados por el lexer.
- Establecimiento de precedencias y asociatividad para operadores aritméticos, relacionales y lógicos, garantizando la correcta evaluación de expresiones complejas.
- Implementación completa de la gramática del lenguaje, incluyendo:
 - Declaraciones de variables globales y locales.
 - Definición de funciones y del procedimiento principal navidad.
 - Manejo de estructuras de control (decide, loop, for).
 - Sentencias de asignación, retorno, lectura y escritura.
 - Manejo de arreglos bidimensionales y su inicialización.
- Construcción del árbol sintáctico, utilizando la clase Nodo.
Cada producción gramatical crea nodos jerárquicos que representan la estructura del programa fuente.
- Gestión de tablas de símbolos con manejo de scopes, coordinada directamente desde el parser:
 - Apertura de nuevos ámbitos al entrar en funciones, bloques y el método principal.
 - Cierre de ámbitos al finalizar cada bloque, imprimiendo su contenido.
 - Registro de símbolos con información de tipo, categoría y posición.
- Integración de acciones semánticas básicas, tales como:
 - Declaración de identificadores.
 - Validación del uso de símbolos previamente declarados.
 - Asociación de tipos a nodos del árbol sintáctico.
- Manejo y recuperación de errores sintácticos en modo pánico:
 - Conteo de errores sintácticos detectados durante el análisis.
 - Reporte de errores indicando línea y columna.
 - Uso de producciones con error y sincronización mediante endl para continuar el análisis.
- Validación final del archivo fuente, determinando si puede o no ser generado por la gramática:
 - Si se detectan errores sintácticos, se notifica al usuario.
 - Solo en ausencia de errores se imprime la tabla de símbolos global y el árbol sintáctico.

TablaSimbolos.java

Se implementó una tabla de símbolos con manejo de scopes, donde cada bloque, función o estructura abre una nueva “rama”. Se implementó de esta forma y no como “hashmap” ya que se concluyó que era más cómodo y se podía abarcar la totalidad de lo solicitado en las especificaciones con mayor eficiencia.

Cada símbolo almacenado contiene información como:

- Nombre del identificador.
- Tipo de dato.
- Categoría (global, local, parámetro, función).

El sistema valida:

- Errores por redeclaración dentro del mismo scope.
- Uso de identificadores no declarados, reportando errores semánticos básicos durante el análisis sintáctico.

Main.java

Coordina la ejecución del sistema:

- Lee el archivo fuente (pruebas2.txt).
- Ejecuta el análisis léxico, generando el archivo tokens.txt con el tipo de token y su lexema.
- Ejecuta el análisis sintáctico, construyendo el árbol sintáctico y gestionando las tablas de símbolos.
- Reporta errores léxicos y sintácticos de forma clara.
- Muestra mensajes de confirmación al finalizar cada fase del análisis.

Construcción del árbol sintáctico

Se construye de forma incremental, siguiendo un modelo apegado al parser, generado por CUP, (que es LALR(1)). Cada reducción crea un nodo que se agrupa a los elementos que se reconocen en el lenguaje. Dentro del manejo en el CUP, se le asigna un literal, ya que no se le puede asignar o fijarse un tipo de dato.

Gestión de la tabla de símbolos por alcance

Se maneja de forma jerárquica, respetando los distintos alcances del programa. Cada bloque, función o estructura de control abre un nuevo scope, en el cual se registran las declaraciones locales. Las búsquedas de identificadores se realizan desde el scope actual hacia los superiores, permitiendo el acceso a variables visibles y detectando errores como redeclaraciones o usos fuera de alcance.

Interacción entre componentes

- **Lexer.flex** → **Lexer.java**: genera el analizador léxico que produce tokens.
- **Parser.cup** → **Parser.java** + **sym.java**: define la gramática y los símbolos que recibe el parser.
- **Parser.java** ↔ **TablaSimbolos.java**: gestiona la declaración y uso de símbolos por scope.
- **Nodo.java**: representa el árbol sintáctico generado.
- **Main.java**: integra todos los componentes y ejecuta el análisis completo del archivo fuente.

Librerías usadas

- **JFlex 1.9.1**
Utilizada para la generación del analizador léxico a partir del archivo Lexer.flex.
Comando de uso: `java -jar ../Librerias/jflex-full-1.9.1.jar Lexer.flex`
- **Java CUP 11b**
Utilizada para la generación del parser y de la clase sym, permitiendo la definición formal de los tokens reconocidos por el analizador léxico.
Comando de uso: `java -jar ../Librerias/java-cup-11b.jar -parser Parser Parser.cup`
- **Bibliotecas Java CUP Runtime**
Incluidas dentro del classpath para el uso de la clase `java_cup.runtime.Symbol` y la interfaz `Scanner`.
- **Java estándar (JDK 8 o superior)**
Utilizado para la compilación y ejecución del proyecto, así como para el manejo de archivos y salida estándar.

Análisis de resultados

Se cumplió con la totalidad del proyecto, incluyendo todos los puntos solicitados, entre los cuales se consideran:

- Reconocimiento completo de todos los tokens definidos en la gramática.
- Generación correcta del archivo `tokens.txt` con tipo y lexema de cada token.
- Implementación de manejo de errores léxicos y sintácticos con recuperación en modo pánico, reportando línea y columna.
- Integración entre JFlex y CUP.
- Construcción del árbol sintáctico para programas válidos.
- Implementación de tablas de símbolos con manejo de scopes, validando declaraciones y usos de identificadores.
- Soporte para estructuras de control, funciones, arreglos y expresiones complejas.
- Uso de GitHub como sistema de control de versiones, manteniendo una bitácora de commits clara.
- Entrega de documentación interna y externa que cumple con los requerimientos del proyecto.

Bitácora

[sebro08/CompiladoresJFlex: Proyecto de compiladores e intérpretes en verano, en JFlex y Cup](#)